

# ENPH 353 Logbook 2024

Robert Liu

## Week #01

**Date:** September 01

**Summary of Activities:** I successfully set up the Linux environment without significant troubleshooting, completing the process in about two hours. To familiarize myself with Linux and terminal usage, I watched the recommended video linked in the first lab document, this video, which helped me better understand the Linux file structure and navigation.

To simplify potential troubleshooting, I retained the default password, ensuring consistency with standard configurations, making it easier for TAs or others to provide support if needed. Additionally, I installed KiCad and other applications to explore their functionality on Linux and compare the experience to Windows usage. I also utilized a USB-C USB rather than a standard one since I did not want to bring adapters everywhere in case I would forget, this did not seem to create any issues with the Linux OS.

## Week #02

**Date:** September 08

**Summary of Activities:** This week, we were tasked with creating a line-following program using OpenCV. The objective was to develop an algorithm capable of detecting and tracking a line from a video feed, with the line closest to the bottom of the frame being highlighted. I began by completing the assigned readings and practice lab to familiarize myself with Python and OpenCV.

Initially, I felt unsure of how to start. I turned to ChatGPT for guidance, using it to outline the basic structure of the program.

My first implementation involved placing a red ball at a fixed  $y$ -coordinate based on the detected line's position. However, I realized that this approach lacked adaptability and accuracy. I then refined the program to analyze multiple  $y$ -coordinates, allowing for a more comprehensive understanding of the line's trajectory. By fitting a curve to these coordinates, I could predict the line's shape and position more accurately.

Despite the improvements, this method proved computationally taxing and occasionally unreliable. To address this, I simplified the algorithm by focusing on a smaller  $y$ -range (200–230 pixels). Within this range, I calculated the average middle point of the line color and placed a red ball at this position for visualization. This approach significantly enhanced the program's efficiency and reliability.

During development, I encountered challenges such as handling missing frames and ensuring smooth transitions in detection. To resolve these issues, I implemented a memory-based fallback mechanism, which used the last known position of the line when detection failed. This ensured robust tracking, even in scenarios where the line was momentarily obscured.

By the end of the week, I had successfully created a functional and efficient line-following program. The program outputs a video with the detected line highlighted and a red ball placed at the calculated position, meeting the lab's objectives. Additionally, I gained valuable experience in leveraging OpenCV's capabilities and applying problem-solving strategies to overcome technical challenges.

Figure 4: Thresholded Frame



Figure 1: Contouring

Figure 7: Path Prediction with Red Ball

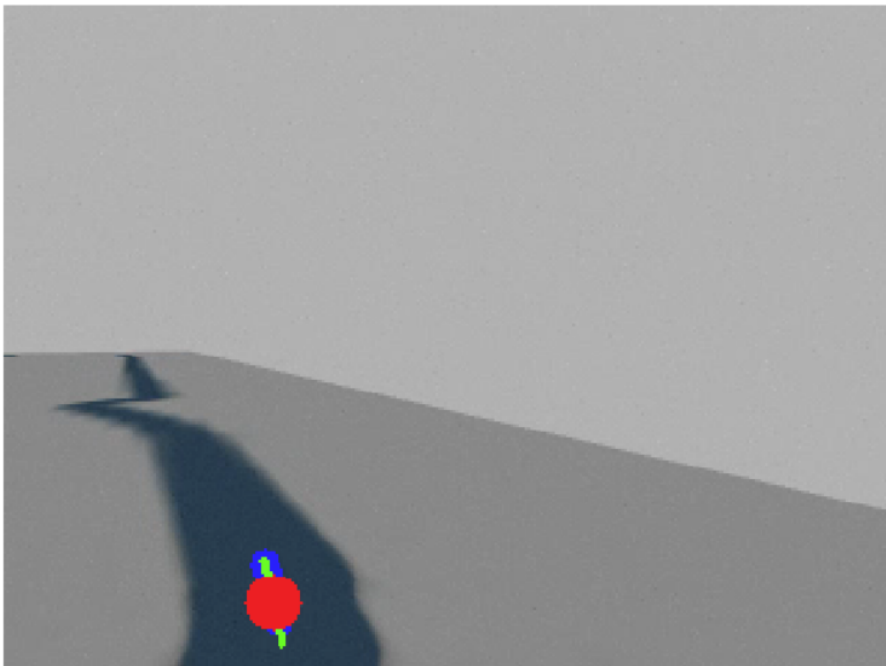


Figure 2: Ball on Road

## Week #03

**Date:** September 15

**Summary of Activities:** This week presented several challenges as I worked to create a robot capable of line-following within the ROS and Gazebo simulation environments.

I initially encountered persistent issues where the robot either floated above the track, failed to appear, or did not respond to movement commands. Additionally, the camera feed often malfunctioned, which hindered progress. These problems were frustrating and time-consuming, especially since this was my first experience with ROS and Gazebo.

To address these issues, I spent time understanding the ROS Publisher/Subscriber model and the interactions between nodes. This helped me debug why the robot wasn't responding. Once the basics were working, I focused on debugging the robot's control logic. Although a simple script successfully moved the robot forward, my camera-based control logic wasn't working as intended. I verified that the `cv_bridge` was functioning correctly and that image data from the camera could be processed in ROS.

I then worked on ensuring that `cmd_vel` commands were published based on real-time image data. Key debugging steps included:

- Checking if image contours were correctly detected.
- Tuning the angular velocity scale factors to improve control.
- Verifying velocity commands using `rostopic echo`.

Simplifications were also explored, such as testing constant forward motion and incrementally reintroducing line-following logic. Systematic testing of image processing, control commands, and robot movement helped pinpoint and resolve issues.

The simplified, less computationally intensive line-following code I developed in Week #02 proved to be a significant advantage this week. It allowed the robot to process image data more reliably and respond to commands in real-time, reducing the lag and instability observed in earlier implementations. This improvement not only made the robot's behavior more predictable but also enabled me to focus on refining the control logic without being hindered by performance issues.

```
1 def image_callback(self, data):
2     try:
3         # Convert the ROS Image message to an OpenCV image
4         frame = self.bridge.imgmsg_to_cv2(data, "bgr8")
5
6         # Convert the frame to grayscale
7         gray_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
8
9         # Apply Gaussian blur to reduce noise
10        blurred_frame = cv2.GaussianBlur(gray_frame, (5, 5), 0)
11
12        # Apply a threshold to isolate the line
13        _, thresh_frame = cv2.threshold(blurred_frame, 100, 255, cv2.
14            THRESH_BINARY_INV)
15
16        # Find contours in the thresholded image
17        contours, _ = cv2.findContours(thresh_frame, cv2.RETR_EXTERNAL,
18            cv2.CHAIN_APPROX_SIMPLE)
19
20        # Initialize lists to store the detected (x, y) points along
21        # the path
22        path_points = []
23
24        # Loop through each y-value to look ahead and find x-
25        # coordinates
26        for fixed_y in self.y_values:
```

```

23     x_values = []
24     for contour in contours:
25         # Filter small contours based on area
26         if cv2.contourArea(contour) < self.min_contour_area:
27             continue
28
29         # Find the bounding box of the contour
30         _, y, _, h = cv2.boundingRect(contour)
31
32         # Check if the contour spans the current y-coordinate
33         if y <= fixed_y <= (y + h):
34             # Approximate the contour to get the points
35             for point in contour:
36                 contour_x, contour_y = point[0]
37                 if contour_y == fixed_y:
38                     x_values.append(contour_x)
39
40         # If two or more x-values (i.e., contours) are found,
41         # calculate the midpoint
42         if len(x_values) >= 2:
43             x_min = min(x_values)
44             x_max = max(x_values)
45             x_mid = (x_min + x_max) // 2
46             path_points.append((x_mid, fixed_y)) # Store the
47             # midpoint
48
49         # If path points are detected, use them
50         if len(path_points) > 2:
51             self.previous_path_points = path_points # Update the
52             # previous path points with the new detection
53             line_detected = True
54         else:
55             line_detected = False
56
57         # If no path points are detected, use the last known points
58         if self.previous_path_points is not None:
59             x_coords, y_coords = zip(*self.previous_path_points)
60
61             # Fit a first-degree polynomial (straight line)
62             poly_coeffs = np.polyfit(y_coords, x_coords, 1)
63             poly_func = np.poly1d(poly_coeffs)
64
65             # Add a red ball at the midpoint of the line at y = 215
66             y_ball = 730
67             x_ball = int(poly_func(y_ball))
68
69             # Control logic: Adjust robot movement based on x_ball
70             # position
71             self.control_robot(x_ball, frame.shape[1], line_detected)
72         else:
73             self.control_robot(None, frame.shape[1], line_detected)
74
75         # Show the processed image (useful for debugging)
76         cv2.imshow("Line Detection", frame)
77         cv2.waitKey(3)
78
79     except CvBridgeError as e:
80         rospi.logerr(f"CvBridge Error: {e}")

```

---

### Listing 1: ROS Image Callback with Line Detection

Here we can see the much more complicated code that I simplified, where there the approach was far to intensive and much simpler options were available.

Later in the week, I accidentally overwrote my code by running an unintended `git pull`, which reset my progress. Fortunately, I had previously reviewed portions of my code with ChatGPT, so I was able to reconstruct the logic relatively quickly. Although tedious, this process reinforced my understanding of the system and allowed me to refine the robot's behavior further.

By the end of the week, I successfully implemented a functional control node that allowed the robot to follow the line. This experience deepened my understanding of ROS, Gazebo, and debugging complex systems.

## Week #04

**Date:** September 22

**Summary of Activities:** This week was significantly smoother compared to last week. I completed the preparatory part of the lab quickly, which allowed me to focus on getting the program functional before integrating my SIFT code. The harder part of this lab was actually understanding SIFT and how homography worked, but once I got a hold and understood what was happening I was able to continue with the code. Initially, there were issues with the layout and functionality of the application. The template image and the camera feed were not properly aligned, and mismatched lines and scaling issues arose. To address these problems:

- I concatenated the template image and the camera feed side by side, ensuring proper alignment.
- Keypoints were rescaled to account for resizing, fixing the mismatched line issue.
- Lines connecting matching keypoints were introduced, but they initially did not align correctly. Adjustments were made by rescaling template keypoints and aligning camera feed keypoints relative to the concatenated display.

To further refine the program, I increased the confidence level for the blue bounding box by raising the threshold for the number of good matches required for homography computation. This ensured the bounding box was retained with stricter validation. Additionally, I:

- Made lines connecting keypoints thinner for better clarity.
- Modified the application to dynamically disable green lines whenever a confident blue bounding box was successfully drawn. This improved the user interface by switching between visualizing keypoint connections and the bounding box based on match confidence.

Here is the main logic of the homography after the improvements:

```
1 # Apply Lowe's ratio test to filter good matches
2 good_matches = []
3 for m, n in matches:
```

```

4         if m.distance < 0.75 * n.distance:
5             good_matches.append(m)
6
7     # Increase the confidence for drawing the bounding box (higher
8     # threshold for good matches)
9     CONFIDENCE_THRESHOLD = 20 # Adjust to increase confidence level for
10    the bounding box
11
12    # If enough matches, we proceed with drawing the bounding box and lines
13    if len(good_matches) >= CONFIDENCE_THRESHOLD:
14        # Extract matched keypoints from the template and frame
15        src_pts = np.float32([self.template_keypoints[m.queryIdx].pt for m
16        in good_matches]).reshape(-1, 1, 2)
17        dst_pts = np.float32([keypoints_frame[m.trainIdx].pt for m in
18        good_matches]).reshape(-1, 1, 2)
19
20        # Compute homography
21        M, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0)
22
23        if M is not None:
24            # Get dimensions of the template image
25            h, w = self.template_image.shape
26
27            # Define points of the template's bounding box
28            pts = np.float32([[0, 0], [0, h-1], [w-1, h-1], [w-1, 0]]).
29            reshape(-1, 1, 2)
30            dst = cv2.perspectiveTransform(pts, M)
31
32            # Draw the bounding box in blue (higher confidence)
33            frame_with_box = cv2.polylines(frame, [np.int32(dst)], True,
34            (255, 0, 0), 3, cv2.LINE_AA)
35
36            # No lines should be drawn when there is a confident bounding
37            # box
38            draw_lines = False
39        else:
40            frame_with_box = frame
41            draw_lines = True # Draw lines if no confident bounding box
42    else:
43        frame_with_box = frame # No bounding box, just the camera frame
44        draw_lines = True # Draw lines if not enough matches for bounding
45        box

```

Listing 2: Applying Lowe's Ratio Test and Drawing Bounding Boxes

While debugging the scaling and alignment issues, I learned the importance of maintaining consistency in the transformation matrices used for homography computation. This lesson reinforced the need to systematically verify every stage of processing, from initial keypoint detection to final visualization.

The polished application successfully displayed the template image on the left and the live camera feed on the right. Depending on the match confidence, it dynamically visualized either green lines connecting keypoints or a confident blue bounding box around the matched object. This dual-mode display ensured clarity for debugging and functionality testing, showcasing the robustness of the implementation.

Furthermore, the refined application was able to handle edge cases where keypoint matches were sparse. By dynamically adjusting thresholds and focusing on reliable

matches, I ensured the application remained functional under less-than-ideal conditions, such as low lighting or partial occlusion of the object. This improvement enhanced its practical usability and reliability.

The improvements in program clarity and functionality were made possible by systematically addressing layout issues early on and iterating on the SIFT matching implementation. My previous experience from debugging ROS in Week #03 also helped streamline the testing process, as I applied similar systematic problem-solving strategies to ensure robustness. This week also highlighted the importance of user interface clarity and adaptive thresholds in creating a functional and user-friendly application.

## Week #05

**Date:** September 29

**Summary of Activities:** This lab was relatively straightforward to implement but took considerable time to fully understand due to the depth and breadth of topics covered. While I was able to complete the required tasks efficiently, understanding the underlying principles presented in the video required additional effort. The video itself was dense, covering a range of foundational concepts related to gradient computation and neural networks. Key topics included:

- **Micrograd Implementation:** The lab introduced Micrograd, a minimal autograd engine, which simplifies gradient computation for neural networks. Implementing basic operations in Micrograd helped me understand how a computational graph tracks operations and calculates gradients step-by-step. This hands-on approach demystified the inner workings of larger frameworks like PyTorch.
- **Backpropagation by Hand:** I performed manual gradient calculations using the chain rule, which is at the heart of backpropagation. This exercise reinforced my understanding of how gradients propagate backward through the network and how weight updates are computed. Breaking down these steps helped clarify the mathematical intuition behind backpropagation.
- **Gradient Resetting:** During implementation, I encountered a common issue where gradients were not reset between training iterations. This oversight led to incorrect updates, as gradients accumulated instead of starting fresh for each backward pass. Correcting this bug provided practical insights into debugging neural networks.

Beyond the implementation tasks, the lab highlighted the importance of understanding different levels of abstraction. Working with Micrograd allowed me to manually inspect each step of the gradient computation process, while contrasting it with PyTorch demonstrated the efficiency and scalability of higher-level tools. By exploring both approaches, I learned the following:

- **Understanding Core Principles:** Micrograd emphasized the need to build a strong foundation by manually constructing and visualizing computational graphs. This experience bridged the gap between theory and practice, helping me understand how automated frameworks like PyTorch function under the hood.



- **Flexibility of Abstraction:** I saw how low-level tools like Micrograd provide granular control over operations, while high-level frameworks like PyTorch automate gradient computation and support custom functions. Understanding both levels of abstraction gave me confidence in debugging and customizing models when needed.
- **Visualization of Gradients:** Drawing out computational graphs and their associated gradients clarified how operations like the chain rule connect different parts of the network. Visualizing these concepts made them easier to understand and apply.

One of the most significant takeaways from this lab was the importance of systematically debugging and verifying every stage of the process. For example, the gradient-resetting issue underscored how small mistakes could lead to cascading errors. Addressing such pitfalls required a methodical approach to tracking errors and testing solutions at each step.

Another critical insight was how foundational knowledge translates to practical applications. By working with a simple tool like Micrograd, I was better equipped to use and understand PyTorch's more advanced features. For instance, I learned how to:

- Use PyTorch to automate gradient computation and leverage its built-in functions for efficiency.
- Reset gradients in PyTorch to avoid issues with accumulation during training iterations.
- Extend PyTorch's functionality by adding custom operations, making it adaptable to specialized tasks.

The lab also highlighted the importance of balancing accuracy and usability in neural network implementations. While Micrograd offered a clear and detailed view of backpropagation, PyTorch's abstractions allowed for rapid development and scalability. This contrast demonstrated how choosing the right tool depends on the task at hand.

By the end of the lab, I had developed a more comprehensive understanding of gradient computation, backpropagation, and the differences between foundational tools like Micrograd and production-level frameworks like PyTorch. This dual perspective not only strengthened my grasp of the basics but also equipped me with the skills needed to troubleshoot and implement more advanced systems. The lab reinforced the value of mastering the fundamentals to ensure success in building and debugging complex machine learning models.

## Week #06

**Date:** October 06

**Summary of Activities:** This lab took longer to understand and prepare the data than to implement the model itself. The most time-intensive part was generating and labeling the data correctly. To ensure proper data generation, I first studied the format of license plates and created random combinations of letters and numbers to simulate them. This required careful attention to detail, particularly when exporting the data. I had to ensure each file was correctly named and bookkept in an Excel file that tracked all filenames. One challenge I faced was ensuring consistency between the files and the

Excel record. Occasionally, if I deleted some files but forgot to update the Excel sheet, the program would encounter errors by attempting to reference files that no longer existed. Debugging this issue required a methodical approach to verify that both the files and the Excel bookkeeping were synchronized. After this system was working reliably, the rest of the implementation proceeded smoothly.

```

1 path = "/content/"
2
3 NUMBER_OF_PLATES = 100
4
5 # Generate a specified number of random license plates in the format "
6   CC NN"
7 # (C: character, N: number)
8 for i in range(NUMBER_OF_PLATES):
9
10     # Generate two random uppercase letters for the plate prefix
11     plate_alpha = ""
12     for _ in range(2):
13         plate_alpha += random.choice(string.ascii_uppercase)
14
15     # Generate a two-digit random number for the plate suffix
16     num = randint(0, 99)
17     plate_num = "{:02d}".format(num) # Ensure the number is zero-
18         padded
19
20     # Load the blank plate template as the base for the generated plate
21     blank_plate = cv2.imread(path + 'blank_plate.png')
22
23     # Convert the OpenCV image to a PIL image to use custom fonts
24     blank_plate_pil = Image.fromarray(blank_plate)
25
26     # Create a drawing context for the PIL image
27     draw = ImageDraw.Draw(blank_plate_pil)
28     # Load a monospaced font for consistent character alignment
29     monospace = ImageFont.truetype(font="/usr/share/fonts/truetype/
30         liberation/LiberationMono-Regular.ttf",
31         size=165)
32
33     # Add the generated plate text to the template image at a fixed
34     position
35     draw.text(xy=(48, 75),
36         text=plate_alpha + " " + plate_num,
37         fill=(255, 0, 0), font=monospace)
38
39     # Convert the modified PIL image back to an OpenCV image
40     blank_plate = np.array(blank_plate_pil)
41
42     # Save the generated license plate image to the specified directory
43     cv2.imwrite(os.path.join(path + "pictures/",
44         f"plate_{plate_alpha}{plate_num}.png"),
45         blank_plate)

```

Listing 3: Generating New Plates

```

1 def slice_image(img):
2     """
3     Slices an input image into four equal-width vertical sections.
4

```

```

5 Parameters:
6     img (numpy.ndarray): The input image to be sliced.
7
8 Returns:
9     list: A list containing four sliced vertical sections of the
10         input image.
11
12 """
13 # Calculate the width of each vertical slice
14 width = img.shape[1] // 4 # Ensure integer division for indexing
15
16 # Slice the image into four vertical sections
17 img1 = img[:, :width] # First quarter (leftmost section)
18 img2 = img[:, width:2*width] # Second quarter
19 img3 = img[:, 2*width:3*width] # Third quarter
20 img4 = img[:, 3*width:] # Fourth quarter (rightmost section)
21
22 # Return the list of sliced images
23 return [img1, img2, img3, img4]

```

Listing 4: Slicing an Image into Four Equal Sections

Once the dataset was prepared, I moved on to processing the images for training. For image detection, I applied a series of preprocessing steps, including filtering the images and dividing them into four separate sub-images to isolate individual characters. These sub-images were then used to train a convolutional neural network (CNN) model. The CNN was designed to recognize 36 possible alphanumeric characters (A-Z and 0-9). Training the model involved some initial challenges, particularly around achieving balanced classes in the dataset. Certain characters appeared more frequently than others, which could bias the training process. To address this, I computed class weights to ensure the model did not disproportionately favor overrepresented characters.

```

1 conv_model = models.Sequential()
2
3 # Add convolutional layers followed by max pooling layers
4 conv_model.add(layers.Conv2D(32, (3, 3), activation='relu',
5                               input_shape=img_shape))
6 conv_model.add(layers.MaxPooling2D((2, 2)))
7
8 conv_model.add(layers.Conv2D(64, (3, 3), activation='relu'))
9 conv_model.add(layers.MaxPooling2D((2, 2)))
10
11 conv_model.add(layers.Conv2D(128, (3, 3), activation='relu'))
12 conv_model.add(layers.MaxPooling2D((2, 2)))
13
14 conv_model.add(layers.Conv2D(128, (3, 3), activation='relu'))
15 conv_model.add(layers.MaxPooling2D((2, 2)))
16
17 # Flatten the output and add dropout for regularization
18 conv_model.add(layers.Flatten())
19 conv_model.add(layers.Dropout(0.5))
20
21 # Add dense layers for classification
22 conv_model.add(layers.Dense(512, activation='relu'))
23 conv_model.add(layers.Dense(36, activation='softmax'))

```

Listing 5: Building a Convolutional Neural Network Model

The CNN architecture itself included multiple convolutional layers for feature extraction, pooling layers to reduce spatial dimensions, and dense layers for classification. I used a learning rate reduction callback to optimize training and ensure steady convergence. Despite these measures, training required fine-tuning, particularly in adjusting hyperparameters like the batch size and the number of epochs. Early in the process, overfitting was an issue, as the model performed well on training data but poorly on validation data. By introducing regularization techniques and using the learning rate reduction callback, I improved the model's generalization ability.

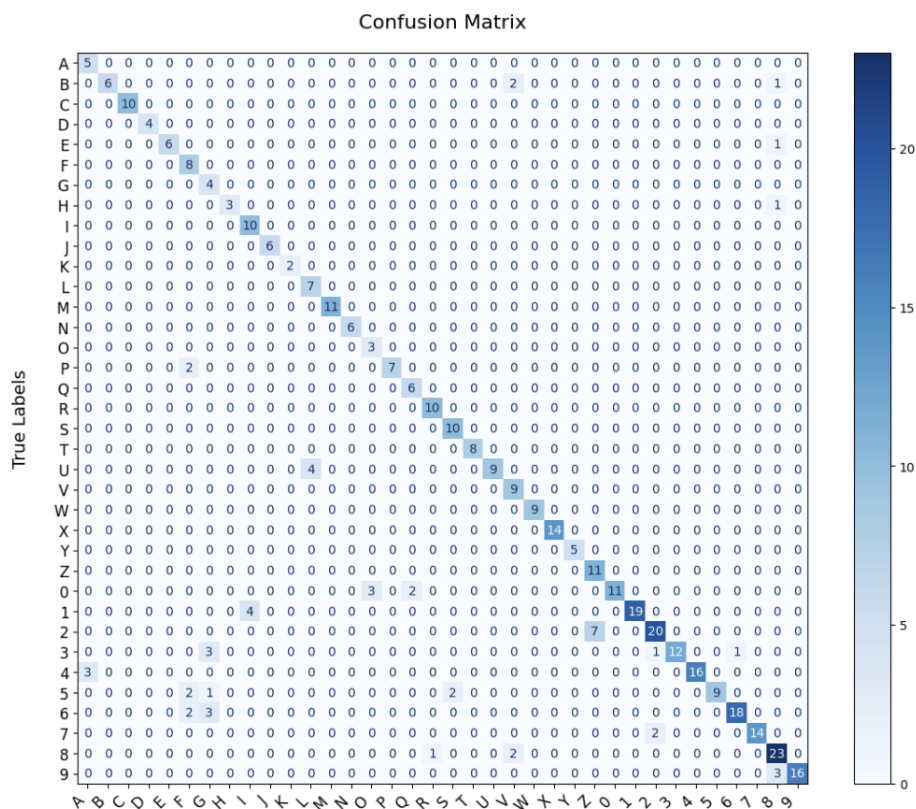


Figure 3: Confusion Matrix

Once the model was trained, I evaluated its performance using confusion matrices. These provided insights into how well the model was recognizing each character and highlighted areas for improvement. For instance, some characters, like 'O' and '0', or 'I' and '1', were frequently misclassified due to their visual similarity. This revealed the need for more robust preprocessing or better training data that could differentiate these ambiguous cases.

Another key challenge was ensuring that the segmented character sub-images were accurately extracted and fed into the model. Minor inaccuracies in the segmentation process could lead to poorly cropped characters, which negatively impacted the model’s accuracy. Debugging this required careful visualization of the segmentation output and making iterative adjustments to the preprocessing pipeline.

Overall, while the implementation of the CNN was straightforward once the data and preprocessing pipelines were set up, the lab underscored the importance of systematic data preparation and debugging. The trained model ultimately performed well, as validated by training and validation loss curves, as well as its accuracy on test data. However,

the process revealed areas for further refinement, such as improving the segmentation pipeline and addressing class imbalances in the dataset. These insights will inform future projects and highlight the importance of attention to detail in data preparation when working with machine learning models.

## Week #07

**Date:** October 13

**Summary of Activities:** This week's setup was incredibly frustrating. No matter what I tried, I could not get the Gazebo simulation to work properly. My robot refused to move, and I spent a significant amount of time troubleshooting. I tried resetting nodes, restarting Gazebo, and even restarting the entire lab multiple times, but nothing worked. Eventually, I decided to revisit the setup from Lab 1 and re-ran all the necessary bash scripts to initialize Gazebo correctly. After this, I was finally able to get the simulation running again. This process highlighted the importance of having a reliable and consistent setup, as unexpected technical issues can severely impact productivity.

The focus of this lab was to implement reinforcement learning using the gym-gazebo framework to train a line-following robot. The task required saving the Q-values whenever the agent achieved a new highest reward during training and loading the best policy at the start of training, if available. Initially, I implemented the logic for saving and loading Q-values directly in the `qlearn.py` file. However, I later moved this functionality to the main driver script for better control and contextual relevance. The updated script tracked the highest cumulative reward during training and saved the Q-values whenever a new record was set. Additionally, it loaded the best Q-values at the beginning of training to ensure progress could resume from the best-known policy. This modification made the workflow much more robust and streamlined.

```
1 # Set the rewards for your action, note that the state divides where
  the line in and is divided in 10
2 if not done:
3     line_in_center = state[4] == 1 or state[5] == 1
4     line_near_center = state[3] == 1 or state[6] == 1
5     line_far_from_center = state[2] == 1 or state[7] == 1
6     line_at_edges = state[0] == 1 or state[1] == 1 or state[8] == 1 or
      state[9] == 1
7
8     if line_in_center: # Line is in the center
9         reward = 15
10        if action == 0: # Moving forward while line is in the center
11            reward += 15
12        elif action in [1, 2]: # Slight left/right actions while line
          is in the center
13            reward -= 5 # Penalize deviation from the center when it's
              already centered
14
15        elif line_near_center: # Line is near the center
16            reward = 10
17            if action == 0: # Moving forward while line is near the center
18                reward += 10
19            elif action in [1, 2]: # Slight left/right actions while line
              is near the center
20                reward += 2 # Encourage slight adjustments to return to
                  the center
```

```

21
22 elif line_far_from_center: # Line is far from the center
23     reward = 5
24     if action == 0: # Moving forward while line is far from the
25         center
26         reward += 5
27     elif action in [1, 2]: # Slight left/right actions to move
28         back to center
29         reward += 10 # Strongly encourage correction
30
31 elif line_at_edges: # Line is at the edges or out of view
32     reward = -10
33     if action in [1, 2]: # Slight left/right actions
34         reward += 15 # Encourage corrective actions
35
36 else:
37     reward = -20 # Penalize if line is lost and no proper action
38     taken
39
40 else:
41     reward = -200 # Penalty for failure state

```

Listing 6: Setting Rewards Based on Line Position and Actions

Despite having the simulation working and the logic implemented, the robot exhibited poor learning behavior. It tended to go straight regardless of the environment, failing to properly follow the line. To address this, I adjusted the reward function to bias the robot toward staying centered on the line. Specifically, I increased the reward multiplier for situations where the line was centered in the robot’s view and where the robot drove straight. Conversely, I penalized the robot more heavily for deviations: if the line was slightly off-center, a smaller penalty was applied, while larger deviations from the center resulted in greater penalties.

Fine-tuning this reward function was a time-intensive process. Small adjustments to the weights significantly affected the robot’s behavior, and it was challenging to find the right balance. Even after considerable experimentation, the robot’s learning was not as effective as I had hoped. While it did demonstrate some improvement, such as staying closer to the line, its overall performance was inconsistent and often suboptimal. This underscored the difficulty of designing effective reward functions in reinforcement learning, where the balance between encouraging desired behavior and discouraging undesirable behavior is critical.

To monitor the robot’s learning progress, I added periodic Q-value saving and updates to reward plots during training. These features provided valuable insights into the agent’s performance and allowed me to make informed adjustments to the training process. However, the robot’s tendency to overfit to certain behaviors and the slow convergence of the Q-values suggested that additional improvements to the reward function or training parameters might be necessary.

Overall, this week’s lab was a mix of troubleshooting technical issues and fine-tuning the reinforcement learning process. While I was able to get the robot to show some learning, the results were far from ideal. This experience reinforced the importance of systematic debugging, thoughtful reward design, and frequent performance monitoring when working with reinforcement learning systems. Moving forward, I plan to explore alternative reward structures and experiment with additional training strategies to improve the robot’s learning capabilities.

## Week #08

**Date:** October 20

**Summary of Activities:** This week's lab introduced deep reinforcement learning through the cross-entropy method, where we trained a neural network to balance a cartpole in the Gazebo simulation environment. While the implementation was manageable, the setup and fine-tuning were particularly challenging, requiring a deep understanding of several tools and concepts.

Initially, I faced significant challenges with the environment setup. Although the cartpole environment was preconfigured, ensuring that Gazebo and ROS were properly initialized took longer than expected. Running the necessary bash scripts to set up the workspace, source the environment, and launch the training script required troubleshooting several minor issues, such as dependencies and file paths. Additionally, I encountered problems with Tensorboard integration, which initially failed to log training data correctly. Debugging these issues reinforced the importance of a systematic approach to verifying software configurations.

The lab focused on the cross-entropy method, a basic yet effective reinforcement learning approach. Unlike Q-learning from previous labs, this method replaces the Q-table with a neural network to predict actions based on states. For this task, the agent observed the cart's horizontal position and the pole's angle relative to vertical. The network's output determined whether to increase or decrease the cart's speed. Training involved running episodes, rewarding actions that kept the pole balanced, and iteratively training the network on elite episodes with the highest rewards. This iterative refinement allowed the agent to progressively learn better policies.

- **Neural Network Design:** Using PyTorch, I implemented a fully connected network with one hidden layer and a ReLU activation function. The cross-entropy loss function was used to optimize the network, combined with the Adam optimizer. Initially, selecting appropriate hyperparameters such as the learning rate and hidden layer size required experimentation to achieve stable training.
- **Reward Function:** Rewards were given for every step the pole remained upright within a 10-degree threshold. This sparse reward structure made it crucial to train the agent on elite episodes effectively. Balancing the top X% of episodes for training was key to ensuring the network learned from successful attempts without overfitting to specific scenarios.
- **Training Process:** Each training iteration involved running several episodes, ranking them by total reward, and using the top-performing episodes to update the network. Monitoring this process through Tensorboard provided insights into the agent's performance over time, such as reward trends and loss curves.

Despite following the prescribed steps, the training process presented additional challenges. Early iterations often resulted in the agent taking random, ineffective actions due to the untrained network. Even after training, the agent sometimes struggled with stability, failing to maintain balance consistently. Adjusting hyperparameters, such as the percentage of elite episodes and the learning rate, was necessary to improve performance. Additionally, while Tensorboard was helpful for visualizing progress, interpreting the results and identifying why certain episodes underperformed required careful analysis of the reward structure and network predictions.

One of the key lessons from this lab was the interplay between reinforcement learning and neural networks. While the cross-entropy method is simple compared to other approaches, it highlighted the importance of balancing exploration (random actions) and exploitation (learning from elite episodes). Furthermore, integrating PyTorch and Tensorboard into the workflow provided valuable experience with industry-standard tools for deep learning and reinforcement learning.

By the end of the lab, the agent demonstrated significant improvement in balancing the cartpole, though its performance was still sensitive to initial conditions and minor perturbations. This underscored the iterative nature of reinforcement learning, where continual tuning and evaluation are essential for achieving robust results. Moving forward, exploring more advanced methods such as policy gradients or deep Q-networks could help address the limitations encountered in this lab.

## Competition Quick Summary

**Summary of Activities:** Worked on time trials and worked on homography for competition, was able to get the hardware set up on my laptop and was trying to generate my training models to use to train with. Most of this was simply used from Lab 6 where we worked on a very similar problem. Also getting a hang of what was going on with the competition setup took a while and knowing which nodes did what and where to send the answers that I had gotten took a while to understand, but once that was figured out I was off to work on the homography and clue detection. We decided to work disjointly in a divide and conquer sort of method hoping that it would reduce the amount of time that we spend on overlapping tasks and be able to be as efficient as possible. We did encounter some errors throughout the time, however in the end we were able to create a reliable and working robot that unfortunately hit the first pedestrian as we did not implement detection for it and the car in time. Note that more information regarding what was done and hiccups will be discussed on the final report.